

文物图像修复项目（扩散模型 Inpainting + 可视化对比网格）

版本：v0.1.4

环境准备

- 创建环境
 - `conda create -n heritage-inpaint python=3.10 -y`
 - `conda activate heritage-inpaint`
- 安装 PyTorch（CUDA 12.6 对应 cu121）
 - `pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121`
- 安装依赖
 - `pip install -U diffusers transformers accelerate safetensors peft opencv-python pillow numpy`

本地默认运行方式

当前脚本已经按本地部署做了默认配置。准备好目录后，在项目根目录直接运行即可：

```
1 | python scripts\infer_inpaint.py
```

默认行为：

- 读取 `imgs/` 中的所有原图。
- 读取 `masks/` 中同名掩码，启动时自动生成反码到 `masks_inverted/`。
- 实际推理默认使用 `masks_inverted/`，并按 `--mask_mode black` 处理，即黑色区域为修补区、白色区域为保留区。
- 输出保存到 `outputs/batch/`，文件名为 `<原文件名>_collage.png`。
- 默认使用 `--steps 30 --guidance 5.0 --size 512 --rows 1 --seed 1234`。

模型和权重放置位置：

- 基础模型放在 `model/models--stabilityai--stable-diffusion-2-inpainting/snapshots/81a84f49b15956b60b4272a405ad3daef3da4590/`，该目录下应包含 `model_index.json`、`unet/`、`vae/`、`text_encoder/` 等 `diffusers` 文件。
- UNet 微调权重放在 `weights/unet_partial_tuned.safetensors`。
- 可选 LoRA 权重放在 `weights/lora_unet.safetensors`，运行时加 `--lora weights/lora_unet.safetensors`。
- `model/`、`weights/`、`imgs/`、`masks/`、`masks_inverted/`、`outputs/` 都是本地数据目录，已被 `.gitignore` 忽略，不应提交到仓库。

数据目录约定

- 单张推理：提供 `--image` 与 `--mask` 两个文件路径

- 批量推理：按文件名匹配，目录为：
 - `imgs/` 放原图（支持递归子目录）
 - `masks/` 或 `masks_inverted/` 放掩码（支持递归子目录）
- 掩码为黑白图：黑色=修补，白色=保留（见参数 `--mask_mode`）。本地默认流程会先从 `masks/` 生成反码到 `masks_inverted/`，再用 `masks_inverted/` 的黑色区域作为修补区。
- 若掩码与原图尺寸不同，脚本会用最近邻将掩码对齐到原图尺寸；原图不会被裁剪或填充。

掩码语义与尺寸适配

- 掩码语义通过 `--mask_mode` 指定：
 - `--mask_mode black` 表示你的掩码“黑=修补、白=保留”（推荐）
 - `--mask_mode white` 表示你的掩码“白=修补、黑=保留”
- 本地默认值是 `--mask_mode black`，并默认使用自动生成的 `masks_inverted/`。如果结果看起来修补区域反了，优先检查 `masks/` 原始掩码和 `masks_inverted/` 反码是否符合预期。
- 尺寸适配：为保证管线稳定，内部会将图与掩码按 `--size` 控制的最长边缩放到“宽高为 8 的倍数”的尺寸送入模型，输出再缩回到原图尺寸；最终可视化与保存全部使用原图尺寸，无白边。若 `--size 0`，则按原图尺寸送入模型。

可视化输出（横排三图 × 竖向多行）

- 每行三图横排：左=原图，中=原图+掩码涂白，右=修复结果，三者严格对齐
- 行间与列间可自定义间距：
 - `--collage_spacing_h` 横向间距像素（默认 20）
 - `--collage_spacing_v` 纵向间距像素（默认 20）
- 生成多行对比：
 - `--rows 4` 生成四行结果（可根据需要调整）
 - `--seed 1234` 设置基础随机种子，逐行递增产生不同结果；为 0 或不设则每行使用随机种子

使用示例

- 默认批量（推荐）
 - `python scripts/infer_inpaint.py`
 - 原图放入 `imgs/`，原始掩码放入 `masks/`，文件名保持一致；脚本会自动生成 `masks_inverted/` 并输出到 `outputs/batch/`。
- 单张
 - `python scripts/infer_inpaint.py --image imgs\pic.png --mask masks_inverted\pic.png --output outputs\result.png`

可选参数（按需添加）：

- 文本提示：`--prompt "修补佛像面部，保持原壁画风格与色彩"`（默认空）
- 采样步数：`--steps 30~50`（默认 30）
- 文本引导：`--guidance 4.5~6.5`（默认 5.0）

- 分辨率: `--size 512/768` (默认 512)
- 掩码语义: `--mask_mode white/black` (默认 black, 黑=修补)
- 行数与间距: `--rows 1 --collage_spacing_h 20 --collage_spacing_v 20`
- 输出路径 (单张): `--output outputs\result.png`
- 批量输出目录: `--output_dir outputs\batch`
- 加载 UNet 权重: `--UNET_weights weights\UNET_partial_tuned.safetensors`
- 加载 LoRA 权重: `--lora weights\lora_UNet.safetensors`

参数详解 (完整)

- 核心:
 - `--model`: inpainting 模型名, 推荐 `stabilityai/stable-diffusion-2-inpainting`
 - `--size`: 目标分辨率 (内部做 8 的倍数适配), 推荐 768; 显存少用 512
 - `--steps`: 采样步数, 建议 30-50
 - `--guidance`: 文本引导强度, 建议 4-7
 - `--prompt`: 文本提示, 默认空
 - `--mask_mode`: 掩码语义, `black` (黑=修补) 或 `white` (白=修补)
 - `--rows`: 同一输入生成的行数, 便于对比不同随机种子结果
 - `--seed`: 基础随机种子; >0 时每行递增, 0/未设为随机
 - `--collage_spacing_h` / `--collage_spacing_v`: 横/纵向间距像素
- 单张:
 - `--image` 原图路径, `--mask` 掩码路径, `--output` 输出路径
- 批量:
 - `--batch_imgs_dir` 原图目录, `--batch_masks_dir` 掩码目录, `--output_dir` 输出目录
 - 递归扫描子目录, 支持 `.png/.jpg/.jpeg`
 - 以无扩展名的文件名 (stem) 匹配原图与掩码
 - 输出文件名为 `<stem>_collage.png`

训练 (可选)

- 轻量 LoRA 微调 (快速贴合文物纹理域):
 - `python scripts/train_lora_inpaint.py --data_root train_data --size 512 --batch 1 --accum 4 --lr 1e-4 --steps 1000 --rank 16 --out weights --model stabilityai/stable-diffusion-2-inpainting`
- 使用 LoRA 权重推理:
 - 在命令中加入 `--lora weights/lora_UNet.safetensors`

常见问题与排错

- 输出大小不一致/出现白边:

- 代码已在送入管线前将尺寸调整到 8 的倍数，并在输出时回缩到原图尺寸；若仍异常，检查掩码是否为极端小图或非二值。
- 未遵循掩码修补：
 - 确保 `--mask_mode` 与你的掩码语义一致；黑=修补用 `black`，白=修补用 `white`
 - 掩码务必为黑白图，灰度值会被二值化 ($\geq 128 \rightarrow$ 白, $< 128 \rightarrow$ 黑)
- 批量只处理部分文件：
 - 只处理“文件名一致”的成对数据；检查 `imgs/` 与 `masks/` 的文件名 stem 是否一致
 - 目录会递归扫描子目录，但仅支持 `.png/.jpg/.jpeg`
- 显存不足/运行缓慢：
 - 将 `--size` 降低为 512 或降低 `--steps`；减少 `--rows`

目录结构

- `scripts/infer_inpaint.py` 推理与可视化脚本
- `scripts/train_lora_inpaint.py` 轻量微调脚本
- `model/` 本地基础模型目录（不提交）
- `weights/` 微调权重目录（不提交），默认读取 `weights/unet_partial_tuned.safetensors`
- `outputs/` 推理与可视化输出（网格拼接 `*_collage.png`）
- `imgs/`、`masks/`、`masks_inverted/` 原图、原始掩码与自动生成的反码目录
- `train_data.zip` 训练数据压缩包（如需分享数据）

推荐使用流程

1. 将基础模型放入 `model/models--stabilityai--stable-diffusion-2-inpainting/snapshots/81a84f49b15956b60b4272a405ad3daef3da4590/`。
2. 将微调权重放入 `weights/unet_partial_tuned.safetensors`。
3. 将原图放入 `imgs/`，原始掩码放入 `masks/`，文件名保持一致。
4. 运行 `python scripts/infer_inpaint.py`。脚本会生成 `masks_inverted/`，再批量输出到 `outputs/batch/`。
5. 若需要进一步贴合文物风格，执行轻量 LoRA 微调，再在推理命令中加入 `--lora` 参数对比前后。

尺寸适配与重采样（与代码一致）

- 当前脚本会将输入图与掩码按 `--size` 控制的最长边缩放到“宽高为 8 的倍数”的尺寸送入 inpainting 管线，然后将输出回缩到原图尺寸（高质量插值）。
- 这能保证生成过程稳定，同时最终保存的三图拼接以原图尺寸为基准；但在高频纹理上可能产生轻微插值失真。若要减少失真，建议适度降低 `--size` 或 `--steps` 并观察对比。

当前脚本默认参数（与代码一致）

- 模型：`--model model/models--stabilityai--stable-diffusion-2-inpainting/snapshots/81a84f49b15956b60b4272a405ad3daef3da4590`

- 分辨率: `--size 512`
- 采样步数: `--steps 30`
- 文本引导: `--guidance 5.0`
- 输出路径 (单张): `--output outputs/result.png`
- 文本提示: `--prompt ""`
- LoRA 权重: `--lora None`
- UNet 权重: `--UNET_weights weights/unet_partial_tuned.safetensors`
- 掩码语义: `--mask_mode black` (黑=修补)
- 单张目录: `--image`、`--mask` 默认未设置; 未设置时进入批量模式
- 批量目录: `--batch_imgs_dir imgs`、`--batch_masks_dir masks_inverted`
- 批量输出目录: `--output_dir outputs/batch`
- 行数与间距: `--rows 1`, `--collage_spacing_h 20`, `--collage_spacing_v 20`
- 随机种子: `--seed 1234` (为 0 时每行随机)

模型原理与实现细节

- 核心思想: 扩散模型的“条件去噪”
 - 使用 `StableDiffusionInpaintPipeline` (SD2-inpainting)。输入包含: 原图 `image` 与掩码 `mask_image`。
 - 掩码白=修补区域 (pipeline 内部约定)。未被掩盖的区域保持不变; 被掩盖的区域通过条件扩散生成匹配上下文的内容。
- 条件构成: 文本提示 + 图像/掩码条件
 - 文本提示: 通过 CLIP 文本编码器提供风格与内容引导, 强度由 `guidance_scale` 控制 (典型 4-7)。
 - 图像与掩码: 掩码白色像素处会被修补, 模型在扩散过程中仅对该区域进行生成, 确保背景一致。
- 解析与尺寸处理:
 - 为匹配 UNet 的卷积栅格, 脚本将原图与掩码缩放到“宽、高均为 8 的倍数”输送给模型, 生成后再缩回原图尺寸, 避免白边与形变。
 - 可视化拼接: 横排三图 (原图、原图+掩码涂白、修复结果), 竖向堆叠多行 (不同随机种子), 便于快速对比。
- 训练逻辑 (轻量域适配):
 - 目标: 让 SD2-inpainting 更贴近文物壁画的色彩与纹理分布, 提升修补区域的材质一致性与细节。
 - 数据: 从 `imgs/` 与 `masks/` (或 `masks_inverted/`) 读取配对; 随机增强 (水平翻转等) 提升鲁棒性。
 - 过程:
 1. 原图与掩码缩放到统一训练尺寸 (推荐 512/384/256 视显存而定)
 2. 通过 VAE 将图像与“掩盖后的图像”编码为潜空间 `latents` 与 `masked_latents`
 3. 采样时间步 `t` 并加噪得到 `noisy_latents`

4. UNet 条件去噪：输入为 `[noisy_latents, mask, masked_latents]` 的拼接，输出预测噪声；用 `MSE(loss)` 与真实噪声对齐
5. 仅训练“注意力投影层”（partial UNet）或 LoRA（低秩适配），减少显存占用与优化器状态体积
 - 保存：训练完成保存为 `weights\UNET_partial_tuned.safetensors`（部分层）或 `weights\lora_unet.safetensors`（LoRA）。推理时直接加载，无需重新训练。
- 为何不建议在 8GB 显存下进行 UNet 全量训练：
 - SD2-inpainting 的 UNet 权重体积很大；全量训练时，优化器状态（动量与二阶矩）会显著增加显存，易 OOM。
 - 解决路径：使用“部分层训练”或 LoRA；降低训练分辨率、增大梯度累积（`accum`）、降低学习率，稳定训练。
- 提示词的作用与调参：
 - 提示词可在修补区域引导生成方向；`guidance_scale` 越高，越“听提示词”。过高可能带来过修或风格漂移。
 - 经验：`guidance=4.5-6.5`；先无提示词对比，再加提示词（例如“修补佛像面部，保持原壁画风格与色彩，纹理自然，五官清晰，边缘平滑过渡”）。
- 缓存与磁盘策略：
 - 使用 `--cache_dir` 指向 F 盘目录，或设置 `HF_HOME/HUGGINGFACE_HUB_CACHE/TRANSFORMERS_CACHE` 到 F 盘，避免占用 C 盘空间。

版本说明

- v0.1.2
 - 新增“模型原理与实现细节”章节，说明 inpainting 条件扩散、尺寸与掩码处理、训练目标与流程、调参策略等。
 - 保持脚本与用法不变；推理支持提示词与批量生成网格对比。